

AgentGate Change Report

Add SANITIZE and ASK_USER decision flows · schema_version 0.1 · 2026-08-14

1. Files changed

FILE	CHANGE
app/core/action_schema.py	Added <code>SANITIZED</code> and <code>WAITING_USER</code> to <code>ExecutionStatus</code>
app/executors/router.py	Extended <code>ExecutionRouter.route()</code> with <code>approved / use_sanitized</code> transitions + new <code>decision_to_execution_status()</code> helper
app/domains/guardrail/decision/simple.py	<code>decide()</code> now produces <code>SANITIZE</code> (secret redaction) and <code>ASK_USER</code> (hint-based) decisions
app/domains/agent/services/browser_prototype_agent.py	<code>_skipped_execution()</code> reuses the shared status mapping (no more generic <code>SKIPPED</code> for the new decisions)
app/api/v1/chat.py	Multi-step browser plan guard now rejects on all non-ALLOW decisions (was only <code>BLOCK / NEED_APPROVAL</code>)
tests/unit/test_decision_flows.py	New — 13 tests covering all five decision paths
tests/unit/test_chat_execute_guard.py	+2 tests: browser plans with <code>SANITIZE / ASK_USER</code> steps never reach Playwright

2. What changed & why

- Decision model** — `Decision` already contained all five values and `DecisionResponse` already had `sanitized_payload / sensitive_entities` , so no duplicate fields were added. `ASK_USER` communicates *why/what's missing* through the existing `reasons` list.
- State machine** — followed the existing implicit state machine (`ExecutionRouter` + the browser agent's skipped-result helper) rather than building a new one. `route()` now enforces the full contract: ALLOW executes; BLOCK never executes; NEED_APPROVAL only after `approved=True` ; SANITIZE returns a `SANITIZED` preview unless `use_sanitized=True` , in which case the **sanitized payload** is substituted via `model_copy` — the original unsafe payload is never passed to the executor; ASK_USER returns `WAITING_USER` and never executes (the loop continues when the user's response produces an updated `ActionRequest`).
- Decision triggers** — `decide()` gained priority-ordered paths: `BLOCK` > `SANITIZE` (recursive regex redaction of `sk-...` keys, AWS/GitHub tokens, `password=` -style credentials) > `ASK_USER` (hints: `ambiguous_target` , `missing_target` , `clarification_needed`) > `NEED_APPROVAL` > `ALLOW` . `next_step` now emits `sanitize / ask_user / approval_queue / execute` .
- Executor safety** — fixed the two entry points that would have executed `SANITIZE / ASK_USER` actions: `ExecutionRouter` and the chat multi-step plan guard. `BrowserPlanner` → `BrowserPlan` → `SelectorResolver` → `BrowserExecutor` → `Playwright` is untouched; existing `goto / click / fill / scroll / screenshot / wait` behavior is unchanged. `Gmail/GitHub/local-file` connectors are untouched and inherit the same decision semantics through `run_guarded_action` .
- Audit & traces** — no schema change needed: `audit_logs.decision_json / execution_json` (JSONB) capture the original request, decision, `reasons` , `sensitive_entities` , `sanitized_payload` , and the final execution status; `execution_status` is `String(32)` , which fits `SANITIZED / WAITING_USER` . `ActionTrace` already embeds the decision/execution dicts, so traces like `{"decision": "SANITIZE", "execution_status": "WAITING_USER"}` flow through automatically.

3. Database migration

None required. All new information lives in existing JSONB columns; the status column's `String(32)` accommodates the new values. The database was not manually modified.

4. Tests added / modified

- New** `tests/unit/test_decision_flows.py` (13 tests): ALLOW executes; BLOCK never executes; NEED_APPROVAL pending → approved executes / rejected doesn't; SANITIZE produces a redacted payload and never executes the original (executes sanitized on `use_sanitized`); ASK_USER returns `WAITING_USER` with clarification reasons, then an updated request re-runs the decision and executes; status-mapping coverage; end-to-end `run_guarded_action` short-circuit checks.
- Modified** `tests/unit/test_chat_execute_guard.py` (+2): secret-bearing and ambiguous browser plans are rejected with `SANITIZED / WAITING_USER` before Playwright.

5. Test commands

```
python -m pytest tests/unit/test_decision_flows.py tests/unit/test_chat_execute_guard.py -q
python -m pytest -q # full suite
python -m ruff check app/executors/router.py app/domains/guardrail/decision/simple.py app/api/v1/chat.py
tests/unit/test_decision_flows.py tests/unit/test_chat_execute_guard.py
```

Full suite: **133 passed**, 1 skipped (Playwright-dependent test). The **7 remaining failures are pre-existing** — verified by stashing the changes and re-running: identical failures in `test_audit_model_registration.py` , `test_recovery.py` , and `test_parser_llm.py` (network/LLM-dependent).

6. Remaining limitations

- **No approval UI:** the approval transition is implemented in `route(approved=True)` and tested, but there is no approve/reject endpoint — `GET /approvals` still just lists pending items.
- **No ASK_USER/SANITIZE orchestration loop:** `run_guarded_action` returns the `WAITING_USER` / `SANITIZED` state; a caller (e.g. the chat flow) must build an updated `ActionRequest` and re-run — demonstrated in tests, not wired into a UI/API endpoint.
- **Sensitive detection is rule-based** (regex over payload strings); it covers common secret formats only.

7. Preserved semantic distinctions

`ALLOW` = execute now (may still fail at execution)

`NEED_APPROVAL` = execute only after explicit approval

`ASK_USER` = clarification/information is required before proceeding

`BLOCK` = never execute

`SANITIZE` = modify/sanitize payload before deciding whether to execute

SANITIZE is never conflated with BLOCK, and ASK_USER is never conflated with NEED_APPROVAL.